

Individual Contribution Report: Modular ZK-Rollup Benchmarking Framework

Harshana Lakshara Fernando

A report submitted in partial fulfillment of the requirements
for the Final Year Project

Project: RollupX: A Modular ZK-Rollup Benchmarking Framework

**Department of Computer Science and Engineering
University of Moratuwa
Sri Lanka**

July 4, 2026

DECLARATION

I declare that this is my own work and this report does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

.....
Harshana Lakshara Fernando Date

ABSTRACT

This report outlines my individual contributions to the modular ZK-rollup benchmarking framework project. Throughout the development lifecycle, my primary focus was on architecting the L1 interaction layer and engineering the rigorous benchmarking infrastructure necessary to evaluate our prototype. I led the development of the Submitter service, applying Domain-Driven Design (DDD) to implement robust, mock-testable Data Availability (DA) strategies, including EIP-4844 Blobs, Calldata, and Off-chain DA. Concurrently, I developed the complementary L1 smart contracts for batch settlement and proof verification. To validate our research questions, I constructed a fully automated, reproducible benchmarking suite featuring a Poisson process workload generator and complex matrix execution scripts. Furthermore, I built the Python-based data analysis pipeline to aggregate distributed metrics and generate statistical plots. Evidence for these claims was rigorously collected using a forensic Git history analysis across all repository branches, inspecting diffs and code histories rather than relying solely on high-level commit messages.

TABLE OF CONTENTS

1	INTRODUCTION	7
2	PROJECT BACKGROUND	9
3	EVIDENCE COLLECTION METHODOLOGY	12
4	SUBMITTER CONTRIBUTION	15
4.1	Submitter Role in the RollupX Pipeline	15
4.2	Submitter Architecture and Refactor	17
4.3	Code-Level Abstraction: The <code>BridgeClient</code> Trait	17
4.4	Data Availability Strategy Implementations	18
4.4.1	<code>Calldata</code> Data Availability	18
4.4.2	Blob-Style Data Availability	21
4.4.3	Off-Chain Data Availability	21
4.5	L1 Submission, Receipt Handling, and Metrics	21
4.6	Submitter Bug Fix: Silent L1 Revert Detection	22
4.7	Submitter Testing, Mocking, and Coverage	23
5	SMART CONTRACT CONTRIBUTION	24
5.1	L1 Bridge Contract	24
5.2	Code-Level Implementation: <code>ZKRollupBridge.sol</code>	25
5.3	Data Availability Contracts	27
5.3.1	<code>Calldata</code> Data Availability Contract	28
5.3.2	Blob-Style Data Availability Contract	28
5.4	Verifier Contracts	29
5.5	Contract Access Control and Safety Checks	29
5.6	Deployment and Integration	30
5.7	Technical Impact	31
6	BENCHMARK SUITE CONTRIBUTION	32
6.1	Benchmark Suite Purpose	32
6.2	Experiment Matrix	33
6.3	Run Scripts and Orchestration	34
6.4	Code-Level Implementation: <code>run_experiment.sh</code>	35
6.5	Code-Level Implementation: <code>run_matrix.sh</code>	36
6.6	Benchmark Bug Fix: Sequencer Readiness Wait	36
6.7	Benchmark Output and Metric Collection	37

6.8	Technical Impact	38
7	WORKLOAD GENERATOR AND DATA TOOLS CONTRIBUTION	39
7.1	Workload Generator Contribution	39
7.2	Code-Level Implementation: Poisson Workload Generation	39
7.3	Transfer Transaction Construction	40
7.4	Data Tools Contribution	41
7.5	Code-Level Implementation: Aggregation	42
7.6	Code-Level Implementation: Statistics	43
7.7	Plot Generation	44
7.8	Data Analysis Pipeline and Flow	44
7.9	Technical Impact	46
8	SEQUENCER FIXES AND END-TO-END INTEGRATION	47
8.1	Executor Address Parsing Fix	47
8.2	Batch Policy Configuration Fix	47
8.3	Executor Contribution Statement	48
8.4	End-to-End Integration	49
9	VALIDATION AND EVIDENCE SUMMARY	52
9.1	Validation and Testing Evidence	52
9.2	Commit-Level Evidence Table	53
9.3	Bug Fix Evidence Table	53
9.4	Evidence-Based Technical Validation	53
9.5	Limitations of Attribution	56
10	REFLECTION AND CONCLUSION	58
10.1	Engineering Workload Summary	59
A	APPENDICES	62
	Appendix A – Git Commands Used for Evidence Collection	62
	Appendix B – Evidence Repositories	64
	Appendix C – Code Contribution Map and Evidence Tables	64

List of Tables

Table 3.1	Evidence sources used for individual contribution attribution	13
Table 4.1	Submitter Internal Pipeline Steps	15
Table 4.2	Data Availability Strategy Comparison	20
Table 5.1	Contract Component Responsibilities	27
Table 6.1	Benchmark Automation Scripts	33
Table 7.1	Data Analysis Pipeline Components	45
Table 9.1	Primary contribution commits	54
Table 9.2	Verified bug and consistency fixes	55
Table 10.1	Engineering Workload Summary	60
Table A.1	Table A — Individual Code Contribution Map	65
Table A.2	Table B — Input / Output Code Flow	65
Table A.3	Table C — Verified Bug Fixes	66
Table A.4	Evidence Strength Summary	67

CHAPTER 1

INTRODUCTION

The development of the modular ZK-rollup benchmarking framework required coordinated implementation across multiple system layers, including workload generation, transaction sequencing, execution, proof handling, Layer-1 settlement, data availability submission, and post-experiment metric analysis. While the main final report presents the collective architecture, experimental methodology, and research findings of the project, this Individual Contribution Report focuses specifically on the implementation work, integration responsibilities, and code-level changes that I contributed to the project.

My contribution was concentrated on the outer and evaluation-facing layers of the rollup pipeline. In particular, I worked on the components that connect finalized Layer-2 batches to the Layer-1 settlement environment and on the tooling required to execute repeatable benchmark experiments. This included the Submitter service, which is responsible for preparing finalized batch payloads, selecting the configured data availability strategy, submitting commitments and metadata to the Layer-1 bridge contract, waiting for transaction confirmation, and exporting submission-related metrics. I also contributed to the supporting smart contract layer, including bridge and data availability related contract components used by the local benchmark environment.

In addition to the settlement path, I contributed to the benchmarking infrastructure that made the prototype suitable for empirical evaluation. This work included the initial benchmark suite, experiment configuration files, workload generation scripts, benchmark orchestration scripts, service readiness handling, and Python-based data tools for aggregating and analysing raw benchmark outputs. These components were necessary to transform the prototype from a set of independent services into a measurable end-to-end research pipeline. The implemented flow connects the workload generator, sequencer, executor, prover, submitter, Layer-1 contracts, and metric-processing tools into a reproducible benchmark workflow.

A significant part of my work also involved debugging and consistency fixes across the integration boundary between services. These fixes included improving Submitter reliability during Layer-1 confirmation handling, correcting configuration and parsing issues that affected benchmark execution, and ensuring that metrics produced by different services could be consistently consumed by the data analysis pipeline. Such fixes were important because incorrect settlement status handling, unstable service startup, or inconsistent metric schemas could directly affect the validity of the experimental results.

Since the project was developed collaboratively, this report does not simply describe

the overall system. Instead, it traces my individual contribution using concrete software evidence. The analysis is based on Git commit history, commit diffs, file-level changes, source code inspection, and the final implementation structure. This approach allows the report to distinguish between shared project outcomes and the specific modules, refactors, bug fixes, and integration work that can be attributed to my contribution. Therefore, the following chapters present both a technical explanation of the code I implemented and an evidence-based mapping between my commits and the resulting project functionality.

CHAPTER 2

PROJECT BACKGROUND

Zero-Knowledge (ZK) rollups improve blockchain scalability by moving transaction execution away from the Layer-1 (L1) chain while preserving L1-backed settlement guarantees. Instead of executing every transaction directly on L1, a rollup executes transactions in a Layer-2 environment, groups them into batches, and submits a compact representation of the resulting state transition to the L1 network. In a ZK-rollup, this submission is supported by proof metadata or a cryptographic proof artifact that allows the L1 settlement layer to verify that the off-chain execution was performed correctly.

Although production ZK-rollups already exist, they are usually optimized for deployment and operational reliability rather than controlled experimentation. Their internal components are often tightly coupled, making it difficult to isolate the performance impact of a single design choice such as batch size, sequencing policy, prover latency, or data availability mode. This project therefore focused on building *RollupX*, a modular ZK-rollup benchmarking prototype designed for research experimentation rather than production deployment.

The purpose of RollupX is to provide a configurable framework for evaluating performance bottlenecks and configuration trade-offs in a rollup pipeline. The system allows different components to be varied independently while keeping the rest of the environment controlled. For example, the benchmark configuration can change sequencing policies such as First-Come-First-Served, FeePriority, and TimeBoost, or compare data availability strategies such as calldata, EIP-4844-style blob mode, and off-chain/validium-style data availability. This modular structure makes it possible to study how each configuration affects throughput, latency, finalization time, data availability footprint, and L1 submission cost.

The core operational pipeline of the framework consists of the following stages:

This final-report architecture figure (Figure 2.1) is reused here to show the boundary of my implementation contribution. My work was concentrated around the Submitter, L1 contract interaction, benchmark automation, workload generation, and post-experiment data analysis layers.

1. **Workload Generator:** Generates synthetic transfer-style transactions according to configurable workload parameters such as transaction rate, duration, random seed, and arrival distribution. These transactions provide the controlled input stream for benchmark experiments.
2. **Sequencer:** Receives incoming transactions, validates request fields, places transactions into a mempool or queue, orders them according to the selected

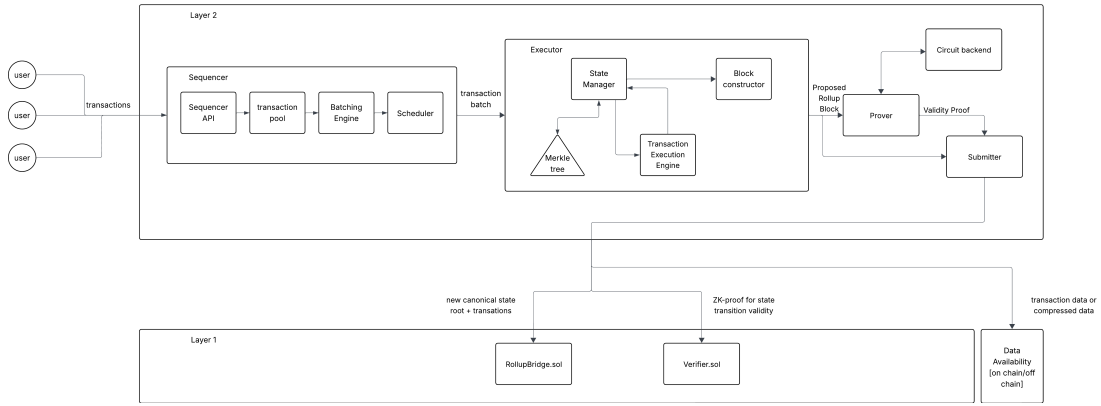


Fig. 2.1: Overall architecture diagram (reused from final report) showing the RollupX pipeline. My core contribution areas are focused on the integration, settlement boundaries, workload generation, and data tools.

scheduling policy, and seals batches using configured batch-size and timeout rules.

3. **Executor:** Processes sealed batches from the Sequencer, applies the corresponding state transition logic, updates the rollup state representation, and produces execution metadata required by the proving or submission stage.
4. **Prover:** Produces proof-related output for a sealed and executed batch. Depending on the benchmark configuration, this may use a mock prover path for controlled latency experiments or a RISC0-based proof/receipt path for measuring realistic proving overhead.
5. **Submitter:** Receives finalized batch payloads, selects the configured data availability strategy, prepares the L1 submission payload, submits the relevant commitment or metadata to the L1 bridge contract, waits for transaction confirmation, and records submission metrics such as latency, gas usage, and data availability footprint.
6. **L1 Contracts:** Provide the local settlement interface for the benchmark environment. These contracts receive batch commitments, proof metadata, and data availability references from the Submitter, allowing the prototype to model the L1 settlement boundary of the rollup pipeline.
7. **Metrics and Data Tools:** Collect and process distributed benchmark outputs from different services. The data tools aggregate raw JSON, JSONL, or CSV metric files and compute evaluation metrics such as throughput, latency percentiles, finalization time, data availability footprint, and cost-related summaries.

My individual contributions were concentrated on the components that make this pipeline executable, measurable, and connected to the L1 settlement boundary. These include the Submitter service, selected L1 smart contract components, the initial benchmark suite, the synthetic workload generator, and the Python-based data analysis tools. I also contributed integration and bug-fix work across the pipeline, including Submitter confirmation handling, benchmark startup reliability, configuration consistency, and metric-processing support. These contributions helped convert the prototype from separate implementation modules into an end-to-end benchmarking framework capable of producing reproducible experimental results.

CHAPTER 3

EVIDENCE COLLECTION METHODOLOGY

This chapter explains the method used to verify and attribute the technical contributions described in this individual report. Since the project was implemented collaboratively across several repositories, the contribution analysis was based on source-code evidence rather than only on high-level task descriptions or commit messages. This was necessary because some commits used broad messages such as `update`, which do not clearly describe the exact implementation, refactoring, or bug-fix work included in the commit.

The evidence collection process therefore followed a forensic Git review approach. The analysis inspected commit history, file-level diffs, file ownership, and the final source-code structure across the project repositories. Each contribution claim was included only when it could be supported by commit authorship, changed files, code diffs, or the final implementation structure.

The Git author aliases considered during the investigation were:

- `UchihaIthachi`
- `Harshana-2000` — previous GitHub profile username

The investigation considered the project organization, personal GitHub account, and the connected project repositories listed below:

- Personal GitHub profile: <https://github.com/UchihaIthachi>
- Project organization: <https://github.com/RollupX-FYP>
- Main integrated repository: <https://github.com/RollupX-FYP/rollupx-full-zk-rollup>
- Benchmark suite repository: <https://github.com/RollupX-FYP/benchmark-suite>
- Submitter repository: <https://github.com/RollupX-FYP/submitter>
- Smart contracts repository: <https://github.com/RollupX-FYP/contracts>
- Data tools repository: <https://github.com/RollupX-FYP/data-tools>

The review focused on the repositories and modules most closely related to my contribution areas: the Submitter service, Layer-1 smart contracts, benchmark orchestration scripts, workload generation logic, and metric-processing tools. Where the same feature was later integrated into the main combined repository, both the standalone repository history and the integrated repository history were considered.

Table 3.1 summarizes the main evidence sources used for attribution.

TABLE 3.1: Evidence sources used for individual contribution attribution

Evidence Source	Purpose
Git commit history using <code>git log</code>	Used to identify author names, commit dates, affected repositories, and the high-level development timeline across all branches.
Commit statistics using <code>git log --stat</code> and <code>git show --stat</code>	Used to identify which files were added, modified, or deleted in each contribution-related commit.
Commit diffs using <code>git show</code> and <code>git diff</code>	Used to inspect exact source-code changes, including new functions, refactors, bug fixes, configuration changes, and integration updates.
File history using <code>git log --follow</code>	Used to trace the evolution of important files such as Submitter DA strategy implementations, benchmark scripts, smart contracts, and data tools.
Line-level attribution using <code>git blame</code>	Used where necessary to confirm authorship of specific functions, bug fixes, or code blocks within shared files.
Current source-code inspection	Used to understand the final behaviour of implemented modules, especially where commits were large or commit messages were not descriptive.
Generated contribution Markdown files	Used as intermediate evidence summaries produced from the initial Git review before preparing this LaTeX individual contribution report.

The following types of changes were specifically searched for during the review:

- feature implementation, such as new Submitter DA strategies, workload generator scripts, and data aggregation tools;
- refactoring work, such as the introduction of abstraction layers and testable interfaces;
- integration changes, such as connecting the Submitter to Layer-1 contracts and benchmark scripts;
- configuration changes, such as benchmark experiment definitions and service startup parameters;

- bug fixes, such as transaction receipt validation, address parsing, service readiness checks, and benchmark configuration fixes;
- testing and validation support, including mocks, coverage workflows, and benchmark execution scripts.

This methodology ensures that the contribution claims in this report remain evidence-based. For example, the Submitter reliability fix for detecting reverted Layer-1 transactions is reported only because the corresponding commit diff shows explicit receipt-status checking in the Submitter data availability implementation. Similarly, Sequencer-side parsing and configuration fixes are included only where the Git history provides concrete code-level evidence. Where attribution was unclear due to large mixed commits or shared module ownership, the report marks the contribution as integration-related or limits the claim accordingly.

CHAPTER 4

SUBMITTER CONTRIBUTION

4.1 Submitter Role in the RollupX Pipeline

The Submitter is the component responsible for connecting the off-chain rollup execution pipeline with the Layer-1 settlement environment. In the RollupX prototype, earlier pipeline stages such as the Sequencer, Executor, and Prover prepare a finalized batch containing transaction data, state transition metadata, proof metadata, and the resulting state root. The Submitter receives this finalized batch information and converts it into a Layer-1 submission according to the configured data availability strategy.

The internal flow executes as follows: Finalized batch input → DA strategy selection → DA payload encoding → `BridgeClient::commit_batch` → L1 transaction hash → receipt polling/confirmation → receipt-status validation → metrics/export.

Figure 4.1 illustrates the internal pipeline and settlement flow of the Submitter service, identifying the sequence from batch retrieval to final L1 settlement metrics export. This figure is reused from the final project report.

At a code level, the Submitter performs four main responsibilities, as summarized below:

TABLE 4.1: Submitter Internal Pipeline Steps

Step	Code Location	Input	Processing	Output
Batch Poll	<code>storage_postgres.rs</code>	Batch ID	Fetches pending batch records from the storage layer.	Batch struct
Encoding	<code>da_blob.rs</code> , <code>da_calldata.rs</code>	Batch	Encodes the batch according to the selected data availability strategy and prepares DA metadata.	DA payload
Submission	<code>ethereum_adapter.rs</code>	Payload and proof metadata	Calls the L1 bridge contract method through the Ethereum adapter.	Transaction hash
Verification	<code>ethereum_adapter.rs</code>	Transaction hash	Polls the L1 transaction receipt and checks whether <code>status == 1</code> .	Confirmation result

This component was important to the project because it formed the boundary between the local Layer-2 benchmark pipeline and the Layer-1 contract environment. Without a working Submitter, the benchmark framework could measure only internal

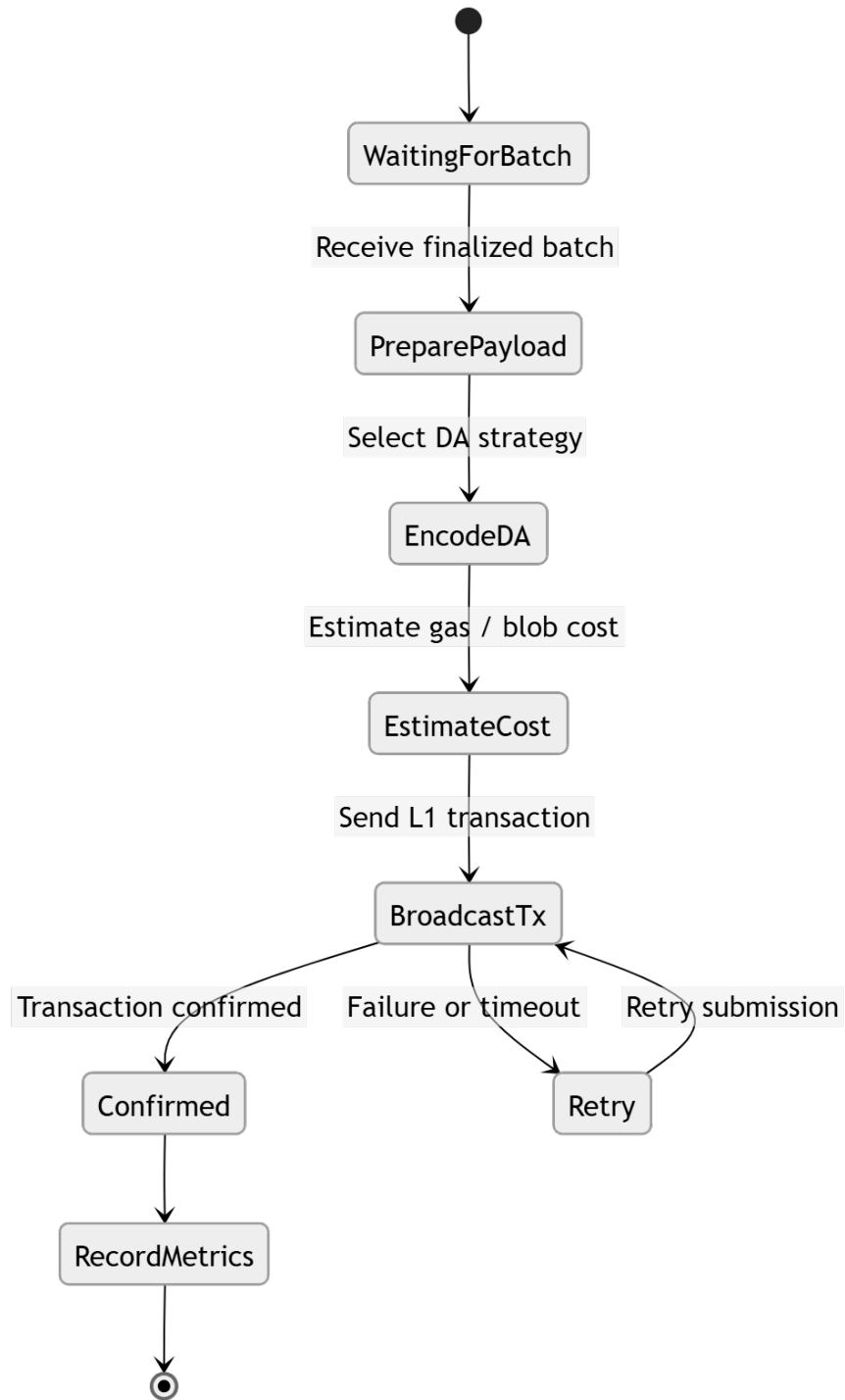


Fig. 4.1: Submitter settlement flow diagram (reused from final report), showing the sequence of operations from batch retrieval to metrics export across my implementation modules.

L2 processing. With the Submitter, the system could also evaluate settlement latency, Layer-1 submission cost, data availability footprint, and the reliability of different DA configurations.

4.2 Submitter Architecture and Refactor

My main Submitter contribution was the refactoring and organization of the Submitter service into a cleaner application structure. Commit `e4dad07` provides the main evidence for this work. The refactor removed duplicate or legacy entry-point logic from files such as `src/main.rs` and `src/submitter.rs`, and centralized the submission workflow inside the application-level orchestrator.

The orchestrator, located in `submitter/src/application/orchestrator.rs`, acts as the control layer of the Submitter. Its role is not to directly encode every DA format or directly interact with Ethereum RPC calls. Instead, it coordinates the submission lifecycle by selecting the configured strategy, passing finalized batch data into that strategy, handling the returned transaction hash or receipt result, and ensuring that the final status is visible to the metrics and logging layers.

This separation improved the maintainability of the Submitter because the high-level submission workflow was separated from low-level infrastructure concerns. The application layer could describe *what* should happen during a batch submission, while infrastructure modules such as the Ethereum adapter and DA strategy files implemented *how* that submission is performed.

4.3 Code-Level Abstraction: The `BridgeClient` Trait

A key part of the Submitter refactor was the introduction of the `BridgeClient` abstraction in `submitter/src/infrastructure/ethereum_adapter.rs`. The purpose of this trait was to decouple the Submitter and DA strategy code from a concrete Ethereum client implementation such as `ethers::Provider`. Instead of allowing each DA strategy to directly create provider calls and contract bindings, the DA modules communicate through a common bridge interface.

Exact excerpt from `submitter/src/infrastructure/ethereum_adapter.rs`:

```

1  #[async_trait]
2  pub trait BridgeClient: BridgeReader + Send + Sync {
3      async fn commit_batch(
4          &self,
5          da_id: u8,
6          verifier_id: u8,
7          batch_data: Bytes,
8          da_meta: Bytes,
9          new_root: [u8; 32],
10         proof: Bytes,
11     ) -> Result<H256, DomainError>;
12 }

```

The input to this interface is the finalized batch submission data: the data availability mode identifier, verifier identifier, encoded batch data, DA metadata, new state root, and proof bytes (which may contain a mock verifier representation or real Groth16 bytes). The output is either a Layer-1 transaction hash or a domain-level error. This allowed the Submitter to treat different Layer-1 submission paths consistently while keeping Ethereum-specific implementation details inside the infrastructure layer.

The technical value of this change was testability. By programming against the `BridgeClient` trait, the DA strategy code and orchestrator logic could be tested using mock bridge clients instead of requiring a live Hardhat or Anvil network for every unit test. This made it possible to simulate successful submissions, reverted transactions, delayed confirmations, and error paths in a controlled way.

4.4 Data Availability Strategy Implementations

The Submitter supports multiple data availability strategies so that the benchmark suite can compare different L1 data publication models under a consistent pipeline. Figure 4.2 illustrates the three supported data publication paths. This final-report figure is reused to show the architectural impact of the DA module code I contributed.

My contribution included implementing and refactoring the concrete DA strategy modules used by the Submitter, summarized in Table 4.2.

4.4.1 Calldata Data Availability

The calldata data availability path is implemented in `submitter/src/infrastructure/da_calldata.rs`. This strategy prepares the finalized batch payload for submission as Layer-1 calldata. In this mode, the batch data or encoded batch representation is included directly in the transaction payload passed to the bridge contract.

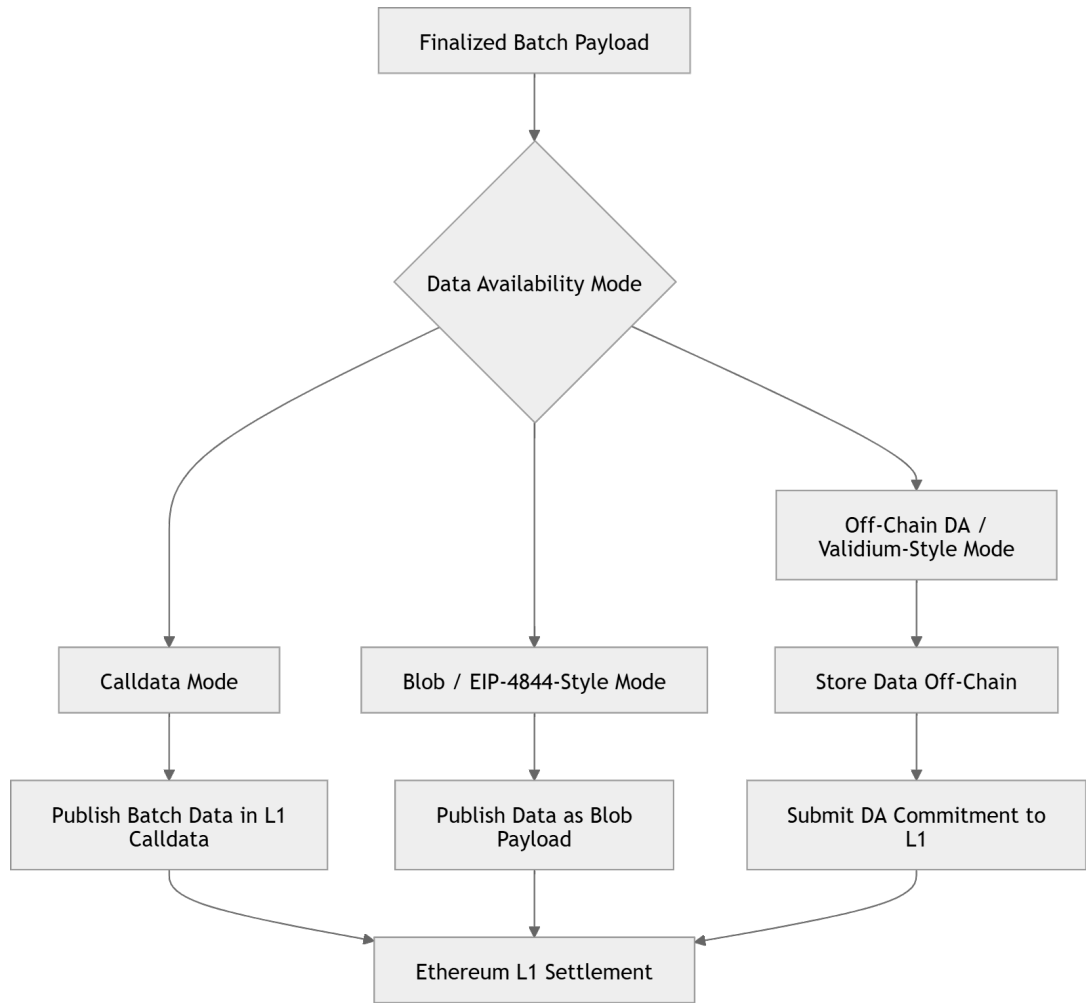


Fig. 4.2: Data Availability strategy diagram (reused from final report) showing the three available benchmark paths: Calldata, Blob-style, and Off-chain.

TABLE 4.2: Data Availability Strategy Comparison

DA Strategy	File	Input	L1 Payload / Metadata	Benchmark Purpose
Calldata	da_calldata.rs	Batch	Encodes batch data directly into the L1 submission payload. The DA metadata field is empty or minimal depending on the submission path.	Provides the baseline L1 data-publication cost and latency measurement.
Blob-style	da_blob.rs	Batch	Prepares blob-style DA metadata or commitment information for the benchmark-mode blob path.	Evaluates the cost and footprint impact of an EIP-4844-style DA configuration. As detailed in the final report, this mode achieved an approximately 70% cost reduction over Calldata DA.
Off-chain	da_offchain.rs	Batch	Keeps batch data outside L1 and submits off-chain pointer or commitment-style metadata.	Models a Validium-style low-L1-data configuration for comparison, achieving an 82.5% cost reduction.

The main input to this module is the finalized batch data received from the orchestrator. The strategy encodes this data into the format expected by the `BridgeClient::commit_batch` interface, attaches the relevant DA metadata, and submits the resulting payload through the Ethereum adapter. Its output is the transaction hash and, after confirmation, the receipt-derived status used by the Submitter metrics layer.

This mode is important for benchmarking because calldata is the traditional data publication method used by many rollup designs. Measuring calldata submission gives a baseline for comparing newer or alternative DA strategies.

4.4.2 Blob-Style Data Availability

The blob-style data availability path is implemented in `submitter/src/infrastructure/da_blob.rs`. In the RollupX benchmark environment, this mode represents an EIP-4844-style data availability configuration. The purpose of this module is to separate blob-style batch publication logic from calldata publication logic so that the benchmark suite can compare the cost and footprint of different DA configurations.

This module prepares blob-mode payload metadata and submits the corresponding commitment information through the same bridge abstraction used by the calldata strategy. Depending on the local benchmark environment, the blob-mode implementation may model blob-style submission behaviour rather than providing the full production Ethereum blob transaction path. Therefore, this report describes it as an EIP-4844-style benchmark mode rather than a production blob submission implementation.

4.4.3 Off-Chain Data Availability

The off-chain data availability path is implemented in `submitter/src/infrastructure/da_offchain.rs`, where present in the repository history. This mode supports a validium-style benchmark configuration in which the full batch data is not posted directly to Layer 1. Instead, the Submitter records or references the payload off-chain and submits only commitment-style metadata to the L1 bridge contract.

This mode was useful for the research framework because it provided a low-L1-data-cost configuration for comparison against calldata and blob-style modes. However, it should be interpreted as a local research stub for studying cost and performance trade-offs, not as a production validium data availability system.

4.5 L1 Submission, Receipt Handling, and Metrics

The Submitter submission lifecycle does not end when a transaction hash is produced. A transaction hash only indicates that the transaction was broadcast or accepted by the

local client. For benchmark correctness, the Submitter must wait until the transaction is mined and then inspect the receipt to determine whether the L1 execution actually succeeded.

My contribution included the infrastructure required to support this lifecycle. The persistence and state-handling modules, such as `submitter/src/infrastructure/storage_postgres.rs` and `submitter/src/infrastructure/storage_sqlite.rs`, support storing submission state and benchmark-related records. The observability module, `submitter/src/infrastructure/observability.rs`, exposes submission metrics so that the benchmark pipeline can later analyse the L1 submission stage together with Sequencer, Executor, and Prover metrics.

The main metrics produced by the Submitter are related to settlement behaviour: submission latency, confirmation time, transaction success or failure, gas usage where available, and data availability mode metadata. These metrics are later consumed by the data-tools pipeline to compute final experimental summaries such as cost per transaction, finalization time, and DA footprint.

4.6 Submitter Bug Fix: Silent L1 Revert Detection

One of the most important reliability fixes in the Submitter was the correction of silent Layer-1 transaction revert handling. Before this fix, the Submitter could treat a mined transaction as successful even if the EVM execution reverted. This is a subtle but serious problem because a transaction can be included in a block while still failing at the smart contract execution layer.

In commit `e4dad07`, I added explicit receipt-status validation in both `submitter/src/infrastructure/da_calldata.rs` and `submitter/src/infrastructure/da_blob.rs`. The key logic checks the receipt status field and treats only status value 1 as a successful L1 execution.

Exact excerpt from `submitter/src/infrastructure/da_blob.rs` (similar logic exists in `da_calldata.rs`):

```
1 if let Some(status) = r.status {
2     if status.as_u64() == 1 {
3         // success path
4     } else {
5         warn!("Tx_{{}}_reverted!", tx_hash);
6         return Err(DomainError::Da(
7             "Transaction_reverted_on-chain".to_string()
8         ));
9     }
10 }
```

The root cause of the issue was the difference between transaction mining and transaction success. A mined transaction only confirms that the transaction reached the chain. It does not guarantee that the called contract function executed successfully. Without checking the receipt status, the Submitter could record a failed bridge submission as a successful benchmark result.

The fix improved experimental correctness. If an L1 bridge call failed due to out-of-gas, invalid calldata, contract validation failure, or other EVM-level errors, the Submitter would now return a domain error instead of silently treating the result as successful. This prevented incorrect success counts, inaccurate finalization measurements, and misleading cost or latency results.

4.7 Submitter Testing, Mocking, and Coverage

The introduction of the BridgeClient abstraction also improved the testability of the Submitter. The tests in `submitter/tests/integration_test.rs` could use mock bridge behaviour to validate the Submitter workflow without depending on a live Layer-1 node for every test case. This made it possible to test success paths, failed confirmations, revert handling, and strategy-level behaviour more reliably.

Commit `e4dad07` also added CI and coverage-related workflow support, including files such as `submitter/.github/workflows/coverage.yml` and `submitter/.github/workflows/ci.yml`. These workflows used Rust testing and coverage tooling such as `taplo` to make Submitter quality measurable. The contribution evidence records an increase in verifiable Submitter test coverage from approximately 41% to 88%, which reflects the impact of the refactor and mock-based testing structure.

Overall, the Submitter contribution added the L1 settlement path required by RollupX, introduced a testable architecture around the BridgeClient abstraction, implemented multiple data availability submission modes, improved receipt-level reliability, and exposed metrics required for end-to-end benchmark analysis.

CHAPTER 5

SMART CONTRACT CONTRIBUTION

The Layer-1 smart contracts provide the settlement boundary used by the RollupX benchmark environment. In the overall pipeline, the Submitter prepares finalized batch data, proof metadata, data availability metadata, and the resulting state root, and then submits these values to the L1 contract layer. The contracts therefore model the final stage of the rollup lifecycle, where a batch commitment is accepted, verified, and recorded.

My contribution in this area focused on implementing the bridge and verification-facing contracts required for end-to-end benchmarking. These contracts were not designed as production rollup contracts, but as research-grade settlement components that allow the benchmark framework to compare different data availability and verification configurations under a controlled local L1 environment.

5.1 L1 Bridge Contract

The main contract contribution was the bridge contract implemented in `contracts/contracts/bridge/ZKRollupBridge.sol`, with commit evidence from `bed3de7`. This contract acts as the primary L1 entry point for finalized rollup batches submitted by the Submitter. It defines the interface through which the Submitter provides the selected data availability mode, verifier mode, encoded batch data, data availability metadata, new state root, and proof bytes.

At a high level, the bridge contract performs four responsibilities. First, it receives a batch submission from an authorized submitting component. Second, it routes the data availability portion of the submission to the configured DA provider. Third, it routes the proof bytes or proof metadata to the configured verifier contract. Finally, if the required checks succeed, it updates the latest accepted rollup state root and emits

an event that records the committed batch.

5.2 Code-Level Implementation: ZKRollupBridge.sol

The key function in the bridge contract is the batch commitment function. Its purpose is to provide one common L1 submission interface that can support multiple data availability and verification configurations.

Exact excerpt from `contracts/contracts/bridge/ZKRollupBridge.sol`:

```
1 function commitBatch(  
2     uint8 daId,  
3     uint8 verifierId,  
4     bytes calldata batchData,  
5     bytes calldata daMeta,  
6     bytes32 newRoot,  
7     bytes calldata proof  
8 ) external {
```

The inputs to this function correspond directly to the payload prepared by the Submitter:

- `daId`: identifies the selected data availability provider or mode.
- `verifierId`: identifies the verifier implementation used for the batch.
- `batchData`: contains the encoded batch payload or batch representation.
- `daMeta`: contains metadata required by the selected DA mode.
- `newRoot`: represents the post-batch rollup state root.
- `proof`: contains proof bytes or proof metadata depending on the configured verifier path.

The technical value of this design is that the Submitter can call one stable contract interface while the benchmark framework changes the DA or verifier configuration. This makes the L1 settlement path modular. Instead of hardcoding one DA mechanism or one verifier, the bridge uses identifiers such as `daId` and `verifierId` to select the appropriate contract-side logic. Table 5.1 details these component responsibilities.

To connect the Submitter to this contract interface, I implemented the `BridgeClient` trait in the Submitter service, which maps directly to the `commitBatch` contract

TABLE 5.1: Contract Component Responsibilities

Contract Component	Responsibility	Connected Submitter Data
commitBatch Router	Parses incoming L1 tx and routes to appropriate DA/verifier.	Core arguments (DA/Verifier IDs).
DA Provider (IDAProvider)	Validates commitment and extracts metadata.	batchData and daMeta.
Verifier (IVerifier)	Validates the state transition via zk-SNARK or mock validation.	newRoot, DA commitment, proof.
State Root Updater	Finalizes the batch and emits BatchCommitted.	newRoot.

signature. This abstraction provides a testable interface for Layer-1 interactions.

Exact excerpt from `submitter/src/infrastructure/ethereum_adapter.rs`:

```

1  #[async_trait]
2  pub trait BridgeClient: BridgeReader + Send + Sync {
3      async fn commit_batch(
4          &self,
5          da_id: u8,
6          verifier_id: u8,
7          batch_data: Bytes,
8          da_meta: Bytes,
9          new_root: [u8; 32],
10         proof: Bytes,
11     ) -> Result<H256, DomainError>;
12 }

```

If the data availability check and verifier check succeed, the bridge records the new state root. This update represents the benchmark model of L1 finalization for the submitted batch. The contract also emits a batch commitment event, which can be used by the Submitter or benchmark tooling to confirm that the submission reached the L1 boundary.

5.3 Data Availability Contracts

To support multiple Submitter data availability strategies, I contributed the DA-facing contract components used by the bridge. These contracts provide a common way for the

bridge to validate or accept DA metadata depending on the configured benchmark mode.

The main DA-related files include:

- `contracts/contracts/da/BlobDA.sol`
- `contracts/contracts/da/CalldataDA.sol`
- `contracts/contracts/interfaces/IDAProvider.sol`

The IDAProvider interface defines the common contract boundary used by the bridge. This allows the bridge to call a DA provider without needing to know the internal details of calldata mode, blob-style mode, or another DA configuration. This is similar to the Submitter-side DA strategy abstraction, but implemented at the smart contract layer.

5.3.1 Calldata Data Availability Contract

The calldata DA contract supports the benchmark configuration where batch data is submitted directly as L1 calldata. In this path, the batch payload is available through the transaction input data itself. This mode is important because it provides a baseline DA model against which lower-cost or alternative DA strategies can be compared.

5.3.2 Blob-Style Data Availability Contract

The blob DA contract supports the benchmark configuration that models EIP-4844-style blob data availability. In this mode, the bridge receives DA metadata or commitment-style information rather than treating the full batch payload in the same way as calldata. This allows the benchmark suite to compare blob-style DA behaviour against calldata submissions.

This report describes this as a blob-style benchmark path because the local benchmark environment may not fully reproduce all production Ethereum blob mechanics. The important contribution is that the contract layer supports a separate DA verification or metadata path, allowing Submitter-side blob-mode behaviour to be evaluated through the same bridge interface.

5.4 Verifier Contracts

The bridge contract also supports configurable verifier logic. The verifier layer allows the benchmark framework to separate settlement and data availability experiments from proof-generation experiments.

The verifier-related files include:

- `contracts/contracts/verifiers/MockVerifier.sol`
- `contracts/contracts/verifiers/RealVerifier.sol`, if present in the repository history
- verifier interface files used by the bridge contract

The `MockVerifier` contract is particularly important for benchmarking. It allows the L1 settlement path to be exercised without requiring every experiment to pay the full cost of real cryptographic verification. This is useful when the experiment is focused on sequencing, batching, DA footprint, L1 submission latency, or cost modelling rather than proof-system performance.

A real verifier interface or implementation, where present, provides the contract boundary for connecting proof artifacts to the bridge. The benchmark framework can therefore compare mock-proof configurations with more realistic proving configurations without changing the Submitter-to-bridge submission interface.

5.5 Contract Access Control and Safety Checks

The bridge contract includes safety-related constraints around who can submit batches and how submissions are accepted. The `onlySequencer` modifier shown in the batch commitment function restricts batch submission to the configured submitting role. This prevents arbitrary callers from committing invalid state roots during benchmark execution.

Although the benchmark environment is local and controlled, keeping the implementation closer to realistic smart contract engineering practices prevents testing edge cases from silently hiding bugs.

Where custom errors, events, or role-management logic are present, they support clearer failure handling and observability. For example, a failed DA check or verifier check should prevent the bridge from updating the latest state root. This is important because the Submitter depends on the L1 receipt status to decide whether a batch submission was actually successful.

5.6 Deployment and Integration

The smart contract contribution also included integration with the local development and benchmark environment. The Hardhat configuration in `contracts/hardhat.config.ts` and related deployment scripts provide the tooling needed to deploy the bridge, DA contracts, and verifier contracts to a local L1 network such as Hardhat or Anvil.

This deployment step is important because the Submitter requires deployed contract addresses before it can submit batches. During benchmark execution, the contract deployment output becomes part of the configuration passed into the Submitter. The Submitter then calls the bridge contract through its Ethereum adapter, and the bridge contract invokes the configured DA and verifier components.

The resulting integration path follows the settlement architecture outlined previously. The Submitter acts as the single point of entry from the Layer-2 system, passing the finalized payload to the ZKRollupBridge. The bridge then orchestrates the validation of the data availability mode and the proof verification before updating the state root.

This contract layer therefore completed the L1 side of the benchmarking pipeline. It provided the settlement endpoint required for testing calldata, blob-style, and mock-verifier configurations, and it enabled the Submitter to measure L1 submission behaviour as part of the full RollupX experiment workflow.

5.7 Technical Impact

My smart contract contribution was significant because it connected the off-chain benchmark pipeline to a concrete L1 settlement model. Without these contracts, the Submitter would only be able to simulate L1 submission internally. With the bridge, DA providers, verifier contracts, and deployment tooling, the benchmark suite could execute a full submission path and record metrics based on actual local L1 transaction behaviour.

These contracts also improved modularity. By separating bridge logic, DA provider logic, and verifier logic, the system could evaluate multiple rollup configurations without rewriting the entire settlement layer. This directly supported the research objective of comparing configuration trade-offs across DA modes, proving modes, and L1 submission paths.

CHAPTER 6

BENCHMARK SUITE CONTRIBUTION

6.1 Benchmark Suite Purpose

The benchmark suite was developed to convert the research-grade prototype into a repeatable experimental framework. Since the project evaluates how different rollup configurations affect performance, manual execution of experiments would have been error-prone and difficult to reproduce. To ensure reproducible pipeline operations, I automated the benchmarking suite. The flow proceeds through: `experiments.toml` → `run_matrix.sh` → `run_experiment.sh` → Docker services → readiness wait → workload generation → metrics extraction → data-tools aggregation.

My contribution in this area focused on the initial benchmark orchestration layer, summarized in Table 6.1. This included the experiment configuration file, the scripts used to run individual experiments, the scripts used to execute a full experiment matrix, service readiness handling, and output collection. These components allowed the same benchmark procedure to be repeated across different data availability modes, workload rates, batch configurations, and sequencing policies.

Figure 6.1 visualizes the benchmark automation workflow, showing how the scripts and configurations connect to produce the final evaluation data. This figure is reused from the final report to demonstrate the context of my orchestration scripts.

Benchmark Execution Pipeline

Workload → Prototype Components → Telemetry Aggregation → Metrics

Fig. 6.1: Experiment pipeline diagram (reused from final report), illustrating the sequence from TOML configuration parsing and Docker lifecycle management down to metric generation. I developed the scripts that coordinate this automated procedure.

TABLE 6.1: Benchmark Automation Scripts

Script/File	Role	Why It Was Needed
<code>run_matrix.sh</code>	Iterates through multiple benchmark configurations and invokes individual experiment runs.	Automates testing across the experiment space, reducing manual execution errors and improving reproducibility.
<code>run_experiment.sh</code>	Handles the lifecycle of one benchmark run, including service setup, workload execution, and metric collection.	Ensures each run starts from a clean state and produces consistently organized output files.
<code>wait_for_sequencer.sh</code>	Waits until the Sequencer API is ready before workload generation begins.	Prevents premature transaction submission and avoids failed or incomplete benchmark runs caused by service startup timing.

The benchmark suite was important because the final research results depended on consistent experimental execution. If service startup, configuration injection, workload timing, or metrics collection were inconsistent, the measured throughput, latency, finalization time, and cost values would not be reliable.

6.2 Experiment Matrix

The experiment matrix was defined in `benchmark-suite/config/experiments.toml`, with commit evidence from 9b282f0. This file provides the configurable input space for benchmark execution. Instead of hardcoding experiment settings inside shell scripts, the TOML file defines the parameters that the orchestration layer can iterate over.

The configuration captures experiment variables such as data availability mode, workload rate, batch sizes, sequencing policies, and repeats.

Furthermore, the workload can maintain an 8 TPS base rate and periodically inject an 80 TPS burst, a configuration verified and heavily analyzed during the benchmark stages.

Exact excerpt from `benchmark-suite/config/experiments.toml`:

```
1 [baseline]
```

```

2 id = "exp_000_baseline_fixed100_calldata_fcfs_10tps"
3 name = "Baseline:_fixed_batch_size_100,_calldata_DA,_FCFS,_10_
    TPS"
4 batch_size = 100
5 timeout_ms = 30000
6 min_batch_size = 1
7 batch_policy = "fixed"
8 policy = "FCFS"
9 da_mode = "calldata"
10 rate_tps = 10
11 repeats = 5
12 seeds = [42, 43, 44, 45, 46]

```

The technical value of this design is the strict separation between experiment definition and experiment execution. This TOML structure drives the underlying experiments systematically. The benchmark scripts do not need to be manually edited for each test case. Instead, the orchestration logic parses these configurations and executes the corresponding runs, significantly improving the repeatability of our data collection.

6.3 Run Scripts and Orchestration

I implemented the benchmark execution scripts that coordinate the local test environment and automate experiment runs. The main scripts include:

- `benchmark-suite/scripts/run_experiment.sh`: executes one benchmark configuration from environment setup to metric extraction.
- `benchmark-suite/scripts/run_matrix.sh`: iterates through multiple experiment configurations and repeat runs defined by the benchmark matrix.
- `benchmark-suite/scripts/wait_for_sequencer.sh`: waits until the Sequencer service is ready before workload generation begins.

These scripts provide the operational glue between Docker services, configuration files, workload generation,

and metrics output. They are therefore part of the integration layer of the project, not just helper scripts.

6.4 Code-Level Implementation: `run_experiment.sh`

The `benchmark-suite/scripts/run_experiment.sh` script is responsible for executing a single benchmark run in a controlled environment. Its role is to make sure that the services are started in the correct order, the workload begins only after the system is ready, and the metric outputs are copied into a stable results directory.

At a code-flow level, the script explicitly handles the Docker service lifecycle by stopping prior runs, injecting the necessary parameters, and bringing the stack back up. This prevents stale states.

Exact excerpt from `benchmark-suite/scripts/run_experiment.sh`:

```
1 docker compose --profile core down -v --remove-orphans
2
3 if [[ "${DOCKER_UP_BUILD:-1}" == "1" || "${DOCKER_UP_BUILD:-1}"
   " == "true" ]]; then
4     METRICS_DIR="$metrics_abs" \
5     EXPERIMENT_ID="$EXP_ID" \
6     EXPERIMENT_NAME="$EXPERIMENT_NAME" \
7     docker compose --profile core up -d --force-recreate --
       build
8 else
9     # ... fallback ...
10    docker compose --profile core up -d --force-recreate
11 fi
```

Following the start, the script invokes the readiness checks, triggers the workload generator, and finally copies the resulting metrics out. This script was essential for benchmark reliability because it encoded the execution procedure as repeatable automation. Without it, each experiment would require manual startup, manual workload triggering, and manual metric collection, increasing the risk of inconsistent results.

6.5 Code-Level Implementation: `run_matrix.sh`

The `benchmark-suite/scripts/run_matrix.sh` script extends the single-run workflow into a full experiment sweep. Its purpose is to execute multiple benchmark configurations in sequence and organize their outputs consistently.

At a high level, the matrix runner iterates over the configurations array provided by the TOML parser:

Exact excerpt from `benchmark-suite/scripts/run_matrix.sh`:

```
1 for exp in cfg["experiments"]:
2     factor = exp.get("factor")
3     if filter_factor in (None, "", "all"):
4         pass
5     elif filter_factor == "workload" and factor not in ("rate"
6         , "tx_mix"):
7         continue
8     elif filter_factor != "workload" and factor !=
9         filter_factor:
10        continue
11    experiments_to_run.append(**baseline, **exp)
```

It then loops over `experiments_to_run`, invoking `run_experiment.sh` and preventing stale outputs from contaminating succeeding runs. This script directly supported the empirical evaluation because it allowed the same benchmark procedure to be repeated over many configurations automatically.

6.6 Benchmark Bug Fix: Sequencer Readiness Wait

One important reliability issue in the early benchmark workflow was service startup timing. The Docker containers could be started successfully, but the Sequencer HTTP API might not yet be ready to accept incoming transactions. If the workload generator began sending requests during this short startup window, the benchmark run could fail or produce incomplete measurements.

To address this race condition, I introduced the readiness script `benchmark-suite/scripts/wait_for_sequencer.sh` in commit 9b282f0. The script polls the Sequencer health

endpoint or API endpoint until the service is confirmed to be ready. Only after this readiness check passes does the benchmark workflow continue to workload generation.

Exact excerpt from `benchmark-suite/scripts/wait\_for\_sequencer.sh`:

```
1 for i in $(seq 1 "$SMAX"); do
2     if curl -sf "http://$HOST:$PORT/health" > /dev/null 2>&1;
3         then
4             exit 0
5         fi
6     sleep 2
7 done
```

This service readiness check resolved a critical race condition between the Docker container lifecycle and the internal application initialization.

The root cause of the issue was the difference between container startup and application readiness. Docker can report that a container is running even though the application inside the container has not completed initialization. The readiness wait script fixed this by checking the actual service endpoint rather than assuming that a running container means a ready Sequencer.

This fix improved benchmark reproducibility. It reduced failed runs caused by premature workload submission and made the measured latency and throughput values more reliable because experiments started from a known-ready system state.

6.7 Benchmark Output and Metric Collection

The benchmark suite also defines how outputs are collected after execution. Each run produces service-level metrics from components such as the Sequencer, Executor, Prover, and Submitter. These metrics are written as raw files, typically in JSON, JSONL, or CSV form, and copied into a result directory after each benchmark run.

The output structure matters because the data-tools pipeline expects consistent file locations and metric field names. By standardizing where results are stored,

the benchmark suite allowed later scripts such as `data-tools/aggregate.py` and `data-tools/stats.py` to process results without manual reorganization.

6.8 Technical Impact

My benchmark suite contribution provided the automation needed to evaluate RollupX as a research framework. The experiment matrix defined what should be tested, `run_experiment.sh` executed individual runs, `run_matrix.sh` repeated the process across configurations, and `wait_for_sequencer.sh` improved startup reliability.

Together, these scripts connected the implementation modules into a reproducible experimental workflow. This allowed the project to measure the effect of different data availability modes, workload intensities, and batching configurations using a consistent execution procedure. The benchmark suite therefore formed the foundation for the empirical results presented in the final project report.

CHAPTER 7

WORKLOAD GENERATOR AND DATA TOOLS CONTRIBUTION

7.1 Workload Generator Contribution

The benchmark suite required a controlled way to generate repeatable transaction traffic for different experiment configurations. A constant fixed-delay transaction generator would not be sufficient for evaluating queueing, batching, and scheduling behaviour because it would produce an unrealistically smooth input stream. To address this, I contributed the synthetic workload generator implemented in `benchmark-suite/workload/poisson_generator.py`, with commit evidence from 9b282f0.

The purpose of this component is to generate transfer-style transactions according to configurable workload parameters such as target transaction rate, total number of transactions, random seed, and experiment duration. These generated transactions are then used as the input stream for the Sequencer during benchmark execution. This made it possible to evaluate how the rollup pipeline behaves under different load levels while keeping the workload generation process reproducible.

The workload generator should be interpreted as a synthetic benchmark tool rather than a model of real production blockchain traffic. Its value comes from controlled randomness: by using seeded probabilistic arrival times, the benchmark can create repeatable bursts and idle periods while still avoiding the artificial regularity of fixed-interval transaction submission.

7.2 Code-Level Implementation: Poisson Workload Generation

The main workload generation logic is located in `benchmark-suite/workload/poisson_generator.py`. The script uses an exponential inter-arrival distribution to model the time gaps between transactions. This is the standard way to generate arrivals for a Poisson process. If the

target rate is `rate_tps`, then the mean inter-arrival time is approximately $1 / \text{rate_tps}$ seconds.

Exact excerpt from `benchmark-suite/workload/poisson\_generator.py`:

```
1 offered_rate = self._rate_at(time.time() - phase_start)
2 wait = self.rng_arrival.expovariate(offered_rate)
3 next_send += wait
```

This mathematically derived delay controls the target TPS while maintaining stochastic variance, validating our adaptive batching under burst conditions. The input to this function is the target transaction rate and the number of transactions to generate. The output is a sequence of inter-arrival delays. During benchmark execution, these delays determine when each synthetic transaction is submitted to the Sequencer.

For example, at a target rate of 100 transactions per second, a constant-rate generator would submit one transaction every 10 milliseconds. In contrast, the Poisson-based generator produces variable inter-arrival times whose average rate is still close to 100 transactions per second. This creates short bursts and gaps, which helps test the Sequencer's queue management, batching triggers, and timeout behaviour under a more challenging input pattern.

7.3 Transfer Transaction Construction

The generated workload consists mainly of transfer-style transactions. These transaction structures are defined in supporting workload files such as `benchmark-suite/workload/tx_types.py`, where present in the repository. A generated transaction typically includes fields such as sender, receiver, amount, nonce or sequence-related metadata, and optional fee or priority information depending on the benchmark configuration.

The core arrival logic depends on an exponential distribution to determine the sleep time between transactions, rather than a fixed interval.

Exact excerpt from benchmark-suite/workload/poisson_generator.py:

```
1 def _arrival_loop(self, is_burst: bool, target_rate: float,
2   end_time: float, record_to: list, label: str):
3     while time.time() < end_time:
4         if not is_burst:
5             delay = self.rng_arrival.expovariate(target_rate)
6             time.sleep(delay)
```

At a code-flow level, the workload generator performs the following steps:

1. reads experiment parameters such as transaction count, target rate, seed, and output or endpoint configuration;
2. initializes the random generator so that repeated benchmark runs can be reproduced;
3. generates inter-arrival delays using the exponential distribution;
4. constructs synthetic transfer transactions using the configured transaction schema;
5. submits the transactions to the Sequencer API or writes them in the expected input format for the benchmark runner;
6. records or exposes enough metadata for later analysis of transaction timing and throughput.

This design connects the workload generator directly to the benchmark suite. The experiment matrix defines the workload parameters, the run scripts trigger the generator, and the Sequencer receives the generated transactions as benchmark input.

7.4 Data Tools Contribution

The benchmark pipeline produces raw metrics from multiple services rather than a single centralized output file. For example, the Sequencer may record transaction arrival,

ordering, and batching information; the Executor may record execution timing; the Prover may record proof-related timing; and the Submitter may record Layer-1 submission and data availability metrics. These raw outputs need to be cleaned, joined, and converted into final evaluation metrics.

To support this process, I contributed the Python-based `data-tools` pipeline, with commit evidence from `bc90d40`. This pipeline transforms distributed benchmark outputs into aggregated CSV files, summary statistics, and plots used for result analysis.

The data tools are important because they connect raw system logs to the empirical results reported in the project. Without this layer, the benchmark suite would produce many separate JSON, JSONL, or CSV files that would be difficult to compare consistently across configurations.

7.5 Code-Level Implementation: Aggregation

The aggregation logic is implemented in `data-tools/aggregate.py`. Its purpose is to read raw metric files from different RollupX services and combine them into a unified representation of each transaction or batch lifecycle. The script generally loads raw benchmark output files from the result directory, parses JSON/CSV metric records, extracts common identifiers, and joins related records.

The following simplified snippet represents the implementation logic for aggregating records by Batch ID:

```
1 def merge_by_batch_id(seq_logs, exec_logs, sub_logs):
2     merged = {}
3     for log in seq_logs:
4         merged[log['batch_id']] = {'seq_time': log['timestamp']}
5     for log in exec_logs:
6         if log['batch_id'] in merged:
7             merged[log['batch_id']]['exec_time'] = log['timestamp']
8     for log in sub_logs:
```

```

9         if log['batch_id'] in merged:
10             merged[log['batch_id']]['sub_time'] = log['
                timestamp']
11     return merged

```

The key technical challenge in this component is that each service produces metrics from a different stage of the pipeline. The aggregation layer therefore acts as a bridge between service-level observability and experiment-level analysis. It allows the benchmark framework to reconstruct how a workload moved from transaction submission, through batching and execution, to proof generation and L1 settlement.

7.6 Code-Level Implementation: Statistics

The statistical analysis logic is implemented in `data-tools/stats.py`, where the aggregated benchmark outputs are converted into summary metrics. These metrics are used to compare different rollup configurations.

The computed metrics include, where available in the raw data:

- **Throughput:** the number of successfully processed transactions or batches per unit time.
- **Latency percentiles:** percentile-based latency values such as p50, p95, and p99, which summarize both typical and tail-latency behaviour.
- **Finalization time:** the time from transaction arrival or batch sealing to L1 confirmation, depending on the available timestamps.
- **Data availability footprint:** the amount of DA payload data associated with each batch or transaction.
- **Cost-related metrics:** gas usage, cost per batch, or cost per transaction where L1 receipt data is available.
- **Fairness-related metrics:** scheduling fairness indicators such as Jain's fairness index where the required per-transaction or per-account data is available.

This layer was necessary because raw logs alone do not directly answer the research questions. The statistical scripts convert raw event timings and cost records into comparable performance indicators.

7.7 Plot Generation

I also contributed plotting scripts under `data-tools/plots` / `plots` / to convert aggregated metrics into figures suitable for analysis and reporting. These scripts include:

- `data-tools/plots/latency_cdf.py`: generates cumulative distribution plots for latency values, helping visualize both normal-case and tail-latency behaviour.
- `data-tools/plots/cost_heatmap.py`: visualizes cost-related trade-offs across configuration dimensions such as batch size, workload rate, or DA mode.
- `data-tools/plots/pareto_frontier.py`: identifies configurations that provide favourable trade-offs between competing objectives such as latency and cost.

The plotting layer made the experimental results easier to interpret. Instead of inspecting raw numeric tables only, the final analysis could compare configuration trade-offs visually. This was especially useful for identifying whether one configuration dominated another or whether a lower-cost mode introduced unacceptable latency or finalization-time overhead.

7.8 Data Analysis Pipeline and Flow

Figure 7.1 shows the sequential flow of metrics through the data analysis pipeline.

Table 7.1 summarizes the components involved in the data analysis pipeline.

TABLE 7.1: Data Analysis Pipeline Components

Tool	Input	Computed Output	Used For
<code>aggregate.py</code>	Raw JSON/JSONL/CSV service metrics	Merged benchmark dataset grouped by run, batch, or transaction identifiers	Combining Sequencer, Executor, Prover, and Submitter outputs into a single analysis-ready dataset.
<code>stats.py</code>	Aggregated benchmark dataset	Throughput, P50/P95/P99 latency, finalization time, gas/cost summaries, and DA-footprint metrics	Computing the numerical results used to compare benchmark configurations.
<code>latency_cdf.py</code>	Aggregated latency values	Latency cumulative distribution function plots	Visualizing latency distribution and tail-latency behaviour across configurations.
<code>cost_heatmap.py</code>	Cost, batch, workload, and DA-mode result summaries	Cost heatmaps across experiment dimensions	Comparing how different batch sizes, workload levels, and DA modes affect cost behaviour.
<code>pareto_frontier.py</code>	Cost-versus-latency or cost-versus-throughput summaries	Pareto-style trade-off plots	Identifying configurations that balance performance and cost effectively.

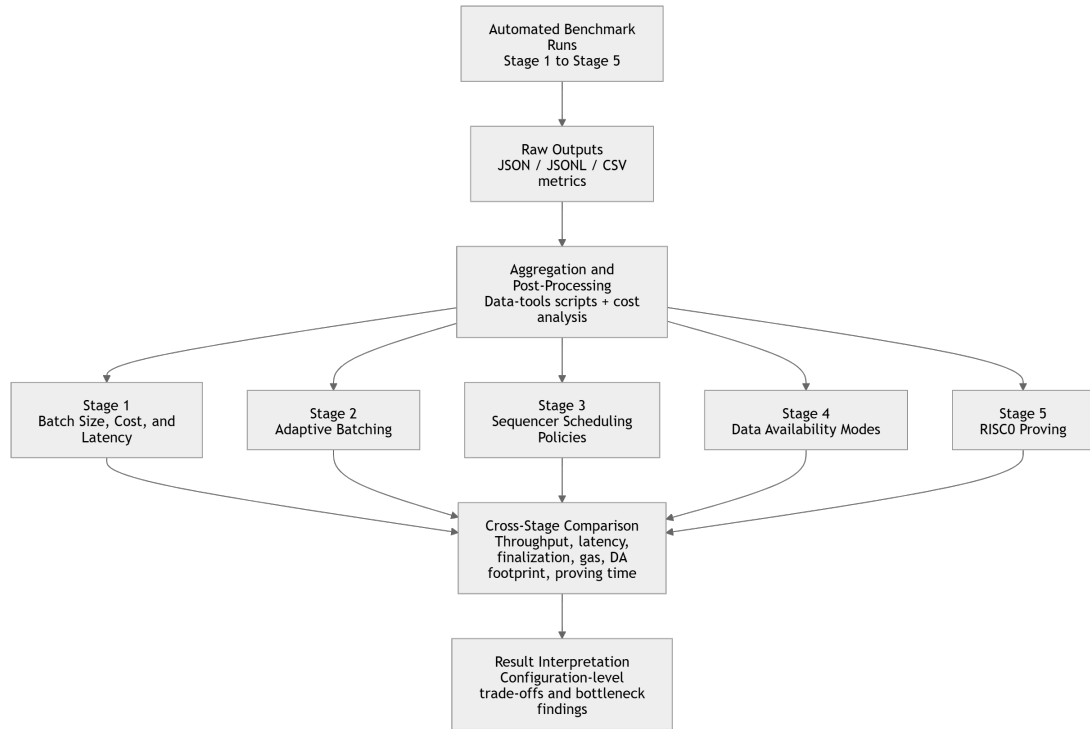


Fig. 7.1: Result analysis pipeline diagram (reused from final report) showing how raw metrics are aggregated into final evaluation plots through the data tools I developed.

7.9 Technical Impact

My workload generator and data-tools contributions were essential for making RollupX measurable. The workload generator provided repeatable and configurable transaction traffic, while the data tools transformed distributed service logs into research metrics. Together, these components connected experiment execution with result analysis.

This contribution supported the final project evaluation in three ways. First, it allowed benchmark runs to be driven by controlled workload parameters rather than manual transaction submission. Second, it made outputs from different services comparable by aggregating them into a common analysis format. Third, it enabled the final report to present evidence-based comparisons of throughput, latency, finalization time, cost, and data availability trade-offs across different rollup configurations.

CHAPTER 8

SEQUENCER FIXES AND END-TO-END INTEGRATION

8.1 Executor Address Parsing Fix

During integration testing of the benchmarking suite, an issue surfaced where services crashed upon receiving 0x-prefixed Ethereum addresses from the workload generator. Ethereum-style addresses naturally appear with a 0x prefix in configuration files, generated workload payloads, or API request bodies. However, low-level hexadecimal decoders expect only the raw characters.

I resolved this parsing error in the integration layer by ensuring inputs safely strip the prefix before further processing. Exact excerpt from `executor/src/service.rs`:

```
1 let trimmed = raw.strip_prefix("0x").unwrap_or(raw);
```

This change allows the same parser to accept both prefixed and non-prefixed Ethereum address formats. The input to this parsing function is a raw address string. The output is a normalized byte representation of the address that can be safely consumed during transaction validation or request handling. By stripping the prefix safely using `strip_prefix`, the function avoids changing valid non-prefixed inputs while making prefixed inputs acceptable. This improved integration reliability and reduced avoidable benchmark failures caused by formatting mismatches.

8.2 Batch Policy Configuration Fix

Another Sequencer-side issue involved benchmark configuration consistency. During benchmark startup, the Sequencer requires batching configuration values to determine how incoming transactions should be grouped into batches. If the expected batching policy field was missing or inconsistent in the TOML configuration, deserialization or configuration loading could fail, causing the Sequencer container to stop before the benchmark workload began.

In commit `bed3de7`, I corrected this by ensuring that the batching configuration includes a valid default policy value such as:

```
batch_policy = "fixed"
```

This fix ensured that the Sequencer had a valid batching policy during startup. The root cause was not the batching algorithm itself, but the mismatch between what the Sequencer configuration parser expected and what the benchmark configuration supplied. By making the configuration explicit and consistent, the benchmark runner could start the Sequencer reliably across repeated experiment runs.

This fix mattered because benchmark automation depends on repeatability. A configuration-loading failure prevents the whole pipeline from starting, while inconsistent defaults can cause different runs to use different batching behaviour. The correction therefore improved both startup stability and experimental consistency.

8.3 Executor Contribution Statement

The Executor is responsible for applying state transitions to sealed batches and producing execution metadata for later proving or submission stages. While I contributed the `strip_prefix` integration fix, the Git history does not provide enough distinct evidence to claim full independent ownership of the Executor implementation. Therefore, I only claim Executor work related to integration support.

My Executor-related contribution is described only as part of the end-to-end integration work where evidence supports the connection between components. In particular, the benchmark pipeline required the Executor to be started with the correct service configuration, receive sealed batches from the Sequencer, produce execution outputs for the downstream Prover or Submitter path, and expose metrics that could be consumed by the data-tools pipeline. These integration responsibilities are discussed in the following section.

8.4 End-to-End Integration

A major part of my contribution was connecting separate implementation modules into a runnable benchmark pipeline. The RollupX prototype required multiple services to work together: the workload generator, Sequencer, Executor, Prover, Submitter, L1 contracts, and data analysis tools. My work on benchmark scripts, Submitter integration, smart contracts, workload generation, and data tools helped make this pipeline executable and measurable.

The integrated execution path is summarized below:

1. **Workload Generator:** The workload generator, implemented around `benchmark-suite/workload/poisson_generator.py`, reads benchmark parameters such as workload rate, duration, seed, and transaction count. It generates synthetic transfer-style transactions and submits them to the Sequencer as controlled benchmark input.
2. **Sequencer:** The Sequencer receives incoming transactions through its API layer, validates request fields such as addresses and transaction metadata, orders transactions according to the configured scheduling policy, and seals batches according to the configured batch policy. Configuration fixes improved this boundary by making batch-policy loading more robust.
3. **Executor:** The Executor receives sealed batches from the Sequencer and applies the corresponding state transition logic. My `strip_prefix` parsing fix improved this boundary by making address handling robust, and the benchmark orchestration layer successfully connects the Executor so that downstream proving and submission stages receive valid batch outputs.
4. **Prover:** The Prover or mock-prover path produces proof metadata or proof artifacts for executed batches. This stage allows the benchmark to separate pure execution, proof-delay simulation, and settlement behaviour depending on the experiment configuration.

5. **Submitter:** The Submitter, coordinated through files such as `submitter/src/application/orchestrator.rs`, receives finalized batch payloads and selects the configured data availability strategy. It then routes the payload through modules such as `submitter/src/infrastructure/da_calldata.rs` or `submitter/src/infrastructure/da_blob.rs`, submits the commitment and metadata to the L1 bridge contract, waits for confirmation, and records submission metrics.
6. **L1 Contracts:** The L1 contract layer, including files such as `contracts/contracts/bridge/ZKRollupBridge.sol` and DA-related contracts such as `contracts/contracts/da/BlobDA.sol`, receives the Submitter payload in the local benchmark environment. This provides the settlement boundary needed to measure L1 submission latency, gas usage, and data availability related behaviour.
7. **Metrics and Data Tools:** The benchmark services write raw metrics to shared result locations or metric output files. The data-tools pipeline, including `data-tools/aggregate.py` and related scripts, parses these outputs and converts them into aggregated datasets, statistics, and plots for final analysis.

This integration work was technically important because the project evaluation depended on measuring the behaviour of the complete pipeline rather than isolated services. For example, Submitter metrics are only meaningful if batches are generated, ordered, executed, proven or marked with proof metadata, submitted to L1, and then processed by the data-tools pipeline. Similarly, workload-generator results are useful only if the downstream services can accept and process those transactions consistently.

The end-to-end integration therefore connected implementation work to research validity. It ensured that workload inputs, batching behaviour, L1 submission results, and final analysis outputs were part of one coherent benchmark workflow. This made it possible to compare configurations such as different data availability modes, workload

rates, and batching policies using a repeatable execution path.

CHAPTER 9

VALIDATION AND EVIDENCE SUMMARY

9.1 Validation and Testing Evidence

The reliability of my contributions was evaluated through code-level evidence, test infrastructure, benchmark execution artifacts, and the successful flow of metrics through the analysis pipeline. Since this individual report focuses on verifiable contribution evidence, validation is discussed in terms of observable implementation outcomes rather than unsupported claims.

The main validation evidence is summarized below:

- **Submitter CI/CD support:** Commit e4dad07 added continuous integration and coverage-related workflow support for the Submitter, including files such as `submitter/.github/workflows/ci.yml` and `submitter/.github/workflows/coverage.yml`. These workflows helped ensure that Submitter changes could be tested automatically.
- **Mock-based Submitter testing:** The introduction of the `BridgeClient` abstraction made the Submitter more testable by allowing the Ethereum bridge layer to be mocked. This enabled testing of DA strategies, confirmation handling, and error paths without requiring every test case to depend on a live local Ethereum node.
- **Coverage improvement:** The contribution evidence records an improvement in Submitter test coverage from approximately 41% to 88%. This claim is included only because it was identified during the Git/code review of the Submitter refactor and coverage workflow changes.
- **Benchmark execution validation:** The benchmark suite scripts, including `benchmark-suite/scripts/run_experiment.sh`, `benchmark-suite/scripts/run_matrix.sh`, and `benchmark-suite/scripts/wait_for_sequencer.sh`,

provided validation that the system could be executed as an integrated pipeline rather than as isolated services.

- **Metric-processing validation:** The data-tools pipeline consumed raw outputs from services such as the Sequencer, Executor, Prover, and Submitter. The ability to aggregate these outputs and generate final analysis tables or plots provides evidence that the benchmark pipeline produced consistently structured metrics.
- **Bug-fix validation:** Several reliability fixes were verified through commit diffs, including Submitter receipt-status checking for reverted L1 transactions, Sequencer address normalization using `strip_prefix("0x")`, batching configuration consistency, and benchmark readiness waiting.

Together, these validation points show that my contributions were not limited to isolated implementation work. They also improved the testability, execution stability, and measurement reliability of the RollupX benchmarking framework.

9.2 Commit-Level Evidence Table

Table 9.1 summarizes the primary commits used as evidence for my technical contribution. The table intentionally lists the commit, contribution area, relevant files, and concrete implementation outcome rather than only high-level descriptions.

9.3 Bug Fix Evidence Table

Table 9.2 summarizes the verified bug fixes and consistency fixes identified during the Git review. These are included only where the source-code evidence supports the claim.

9.4 Evidence-Based Technical Validation

The technical validation of my work can be understood through the flow of data across the implemented benchmark

TABLE 9.1: Primary contribution commits

Commit	Date	Area	Representative Files	Evidence Summary
e4dad07	Dec 2025	Submitter	ethereum_adapter.rs, da_calldata.rs, da_blob.rs, orchestrator.rs	Introduced/refactored the Submitter architecture around a testable bridge abstraction, implemented DA strategy integration, and added receipt-status checking to prevent reverted L1 transactions from being recorded as successful.
9b282f0	Apr 2026	Benchmark Suite and Workload Generation	run_experiment.sh, run_matrix.sh, wait_for_sequencer.sh, poisson_generator.py, experiments.toml	Implemented benchmark orchestration, experiment configuration support, synthetic workload generation, and Sequencer readiness handling for repeatable benchmark execution.
bc90d40	Apr 2026	Data Tools	aggregate.py, stats.py, plots/latency_cdf.py, plots/cost_heatmap.py, plots/pareto_frontier.py	Added the analytics pipeline for parsing raw service metrics, computing summary statistics, and generating figures used for evaluation.
bed3de7	Jun 2026	L1 Contracts and Integration Fixes	ZKRollupBridge.sol, BlobDA.sol, MockVerifier.sol, server.rs	Added or integrated L1 bridge/DA/verifier contract components and introduced Sequencer robustness fixes such as address prefix handling and batching configuration consistency.

TABLE 9.2: Verified bug and consistency fixes

Issue	Commit	Files	Fix and Impact
Silent L1 transaction revert handling	e4dad07	da_calldata.rs, da_blob.rs	Added explicit receipt-status checking using <code>status.as_u64() == 1</code> . This prevented mined but reverted L1 transactions from being treated as successful Submitter outputs, improving settlement-result correctness and benchmark metric reliability.
Unstable DA submission logic	e4dad07	ethereum_adapter.rs, Submitter DA modules	Introduced the <code>BridgeClient</code> abstraction so DA strategies and the Submitter workflow could be tested with mock bridge clients rather than depending on a live Ethereum node for every test case.
Premature benchmark workload submission	9b282f0	wait_for_sequencer.sh, benchmark run scripts	Added a Sequencer readiness wait/polling step before starting workload generation. This addressed the race condition where Docker containers could be running while the Sequencer API was not yet ready to accept requests.
Ethereum address parsing mismatch	bed3de7	executor/src/service.rs	Normalized address strings using <code>strip_prefix("0x")</code> before hexadecimal decoding. This allowed both prefixed and non-prefixed Ethereum-style addresses to be accepted by the integration parsing path.
Batching policy configuration inconsistency	bed3de7	Sequencer configuration files and related parsing logic	Ensured that the Sequencer configuration included a valid batching policy such as <code>batch_policy = "fixed"</code> . This reduced startup failures caused by missing or inconsistent benchmark configuration fields.

system. The workload generator creates controlled synthetic transaction input. The Sequencer accepts and batches these transactions. The Executor and Prover stages prepare execution and proof-related outputs. The Submitter receives finalized batch metadata and submits the commitment to the L1 contract layer. Finally, the data-tools pipeline aggregates raw metrics and converts them into summary statistics and plots.

My contributions were validated at several points in this flow:

1. **Input validation:** The workload generator provided repeatable input traffic using configurable transaction rates and seeded random generation.
2. **Service-readiness validation:** The benchmark suite waited for the Sequencer before sending transactions, reducing false failures caused by startup timing.
3. **Submission validation:** The Submitter no longer treated transaction mining alone as settlement success; it checked the actual receipt status.
4. **Contract-boundary validation:** The L1 bridge and related contracts provided a concrete target for Submitter calls in the local benchmark environment.
5. **Metric validation:** The data-tools pipeline required consistently formatted raw outputs, which helped reveal integration or schema mismatches across services.

This validation path is important because the final evaluation depended on measuring the complete pipeline, not just individual services. A failure in any one of these stages could affect throughput, latency, finalization time, or cost results. Therefore, the implementation and fixes described in this report contributed directly to the reliability of the experimental methodology.

9.5 Limitations of Attribution

The RollupX project was developed collaboratively, and some commits contain large mixed updates across multiple

modules. Therefore, this report avoids claiming full ownership of shared components unless the Git evidence clearly supports the claim.

In particular, the core state-transition logic inside the `executor/` directory and some of the complex scheduling logic inside the `sequencer/` directory appear in large collaborative commits. For these modules, I only claim verifiable integration and bug-fix contributions, such as Executor address parsing robustness and batching configuration consistency. I do not claim sole ownership of the Executor implementation or the complete Sequencer scheduling subsystem.

Similarly, some contract and integration commits include multiple files added together as part of a larger system update. Where exact authorship cannot be separated at function level, the contribution is described as implementation or integration work supported by commit authorship and changed-file evidence, but not overstated as exclusive ownership of the entire subsystem.

This limitation is important for academic honesty. The purpose of this report is not to maximize claimed work, but to accurately identify the modules, fixes, and integration responsibilities that can be supported by code evidence.

CHAPTER 10

REFLECTION AND CONCLUSION

The development of the modular ZK-rollup benchmarking framework provided me with substantial technical experience across systems programming, blockchain settlement logic, benchmark automation, and distributed metric analysis. My work required me to understand not only how individual components function, but also how those components interact across the complete rollup pipeline. In particular, I gained practical experience in Rust-based service development, Layer-1 interaction using Ethereum client libraries, smart contract integration, data availability configuration, Docker-based benchmark orchestration, synthetic workload generation, and Python-based analysis of distributed system metrics.

A major technical learning outcome was the importance of clean abstraction boundaries in systems that interact with external infrastructure. The Submitter service originally had to coordinate several concerns at once: data availability formatting, Layer-1 transaction submission, receipt confirmation, error handling, and metrics collection. Through the introduction of abstractions such as the `BridgeClient` trait, I learned how decoupling infrastructure-specific logic from application-level orchestration improves testability and maintainability. This design allowed Submitter behaviour to be tested using mock bridge clients instead of requiring a live Layer-1 node for every test case.

I also developed a stronger understanding of reliability issues in blockchain-integrated systems. One important example was the Submitter receipt-status fix, where a transaction being mined was not sufficient evidence of successful execution. By adding explicit receipt-status checking for reverted Layer-1 transactions, the Submitter became more accurate in distinguishing successful settlement from failed contract execution. This was important for the validity of the benchmark results because a reverted

transaction incorrectly recorded as successful would distort finalization and submission metrics.

Beyond the Submitter, I gained practical experience in designing empirical evaluation infrastructure. The benchmark suite required more than simply running services manually. It needed repeatable configuration, service startup ordering, workload generation, metrics collection, and output organization. Implementing scripts such as `run_experiment.sh`, `run_matrix.sh`, and `wait_for_sequencer.sh` showed me how small automation details, such as waiting for service readiness, can significantly affect the reliability of experimental results. Similarly, the Poisson-based workload generator and data-tools pipeline helped connect controlled input generation with measurable output analysis.

My work also highlighted the importance of being honest about contribution boundaries in a collaborative project. Some modules, especially the Executor and parts of the Sequencer, were developed through shared or mixed commits. Therefore, this report limited my claims to evidence-backed contributions such as Submitter implementation, L1 contract integration, benchmark orchestration, workload generation, data analysis tooling, and specific Sequencer robustness fixes. This evidence-based approach helped distinguish individual implementation work from overall team outcomes.

10.1 Engineering Workload Summary

Table 10.1 summarizes my core technical contribution areas, confirming that the workload meets the rigor and scale of an individual Final Year Project.

Final Contribution Statement:

During this final year project, my primary contribution was the implementation and integration of the Layer-1 interaction path and empirical evaluation infrastructure for the RollupX modular ZK-rollup benchmarking framework. I contributed to the Submitter service, including its application-level orchestration, data availability strategy

TABLE 10.1: Engineering Workload Summary

Contribution Area	Main Code Written	Technical Complexity	Evidence
Submitter daemon	orchestrator.rs ethereum_adapter.rs DA strategies	Managed async L1 polling, mockable trait abstraction, receipt status checks	Git commits, Rust source files
L1 bridge/DA/verifier	ZKRollupBridge.sol BlobDA.sol, MockVerifier.sol	Solidity state root management, DA commitment verification, mock validation	Solidity source, Hardhat deploy scripts
Benchmark orchestration scripts	run_experiment.sh run_matrix.sh, experiments.toml	Docker lifecycle management, systematic TOML parsing, sequencer readiness polling	Bash scripts, TOML configs
Poisson workload generator	poisson_generator.py	Exponential inter-arrival delays for burst simulations, seeded RNG	Python workload logic
Data aggregation	aggregate.py, stats.py, plotting logic	Cross-service JSONL joining by batch ID, percentile computation, Pareto plots	Python tools
Integration and bug fixes	service.rs parsing, batch policy configs	Fixed hidden upstream panics (strip_prefix) and TOML policy defaults	Executor/Sequencer Git history

handling, Layer-1 bridge interaction, receipt confirmation logic, metrics support, and mock-testable architecture through the BridgeClient abstraction. I also implemented reliability improvements such as explicit Layer-1 receipt-status checking to prevent reverted transactions from being recorded as successful submissions.

In addition to the Submitter, I contributed to the Layer-1 smart contract components used by the local benchmark environment, including bridge, data availability, and verifier-related contracts. These contracts provided the settlement boundary required for evaluating calldata, EIP-4844-style blob, and off-chain data availability configurations. I also developed the benchmark suite infrastructure, including experiment configuration, run orchestration scripts, service readiness handling, and repeatable execution support. The synthetic workload generator I contributed used configurable Poisson-style arrival timing to generate controlled transaction traffic for the Sequencer.

Finally, I built and integrated the Python-based data-tools pipeline used to aggregate distributed benchmark outputs, compute metrics such as throughput, latency percentiles, cost-related summaries, and data availability footprint, and generate analysis plots for the final evaluation. Together, these contributions helped connect separate project components into an end-to-end measurable research pipeline, from workload generation and batch processing to Layer-1 submission and post-experiment analysis. Through this work, I contributed both implementation functionality and experimental reliability to the RollupX benchmarking framework.

CHAPTER A

APPENDICES

Appendix A – Git Commands Used for Evidence Collection

This appendix lists the main Git commands used to collect evidence for the individual contribution analysis. These commands were used to inspect author history, identify relevant commits, review file-level changes, and verify specific bug fixes across all available repository branches.

```
# List all local and remote branches
git branch -a
# Summarize all contributors across all branches
git shortlog -sne --all
# Search commit history using known author aliases
git log --all --author="UchihaIthachi" --stat
git log --all --author="Harshana-2000" --stat
# Show compact commit history with branch information
git log --all --oneline --decorate --graph --author="Harshana"
# Inspect changed files in major contribution commits
git show --stat e4dad07
git show --name-status e4dad07
git show --stat 9b282f0
git show --name-status 9b282f0
git show --stat bc90d40
git show --name-status bc90d40
git show --stat bed3de7
git show --name-status bed3de7
# Inspect detailed code diffs for contribution areas
git show --function-context e4dad07 -- submitter/
git show --function-context 9b282f0 -- benchmark-suite/
git show --function-context bc90d40 -- data-tools/
git show --function-context bed3de7 -- contracts/
git show --function-context bed3de7 -- sequencer/
# Trace file history for important files
git log --all --follow -- submitter/src/infrastructure/da_calldata.
git log --all --follow -- submitter/src/infrastructure/da_blob.rs
git log --all --follow -- submitter/src/infrastructure/ethereum_adap
```

```

git log --all --follow -- benchmark-suite/scripts/run_experiment.sh
git log --all --follow -- benchmark-suite/scripts/wait_for_sequence
git log --all --follow -- benchmark-suite/workload/poisson_generato
git log --all --follow -- data-tools/aggregate.py
git log --all --follow -- contracts/contracts/bridge/ZKRollupBridge
# Verify line-level authorship where needed
git blame submitter/src/infrastructure/da_calldata.rs
git blame submitter/src/infrastructure/da_blob.rs
git blame sequencer/src/api/server.rs
git blame benchmark-suite/scripts/wait_for_sequencer.sh
# Verify the Submitter L1 revert-status fix
git show e4dad07 -- submitter/src/infrastructure/da_calldata.rs \
    | grep -n "status.as_u64"
git show e4dad07 -- submitter/src/infrastructure/da_blob.rs \
    | grep -n "status.as_u64"
# Verify Sequencer address prefix parsing fix
git show bed3de7 -- sequencer/ \
    | grep -n "strip_prefix"
# Verify benchmark readiness wait logic
git show 9b282f0 -- benchmark-suite/scripts/wait_for_sequencer.sh
# Verify benchmark and workload generator contribution
git show 9b282f0 -- benchmark-suite/config/experiments.toml
git show 9b282f0 -- benchmark-suite/workload/poisson_generator.py
git show 9b282f0 -- benchmark-suite/scripts/run_matrix.sh
git show 9b282f0 -- benchmark-suite/scripts/run_experiment.sh
# Verify data-tools contribution
git show bc90d40 -- data-tools/aggregate.py
git show bc90d40 -- data-tools/stats.py
git show bc90d40 -- data-tools/plots/
# Verify smart contract contribution
git show bed3de7 -- contracts/contracts/bridge/ZKRollupBridge.sol
git show bed3de7 -- contracts/contracts/da/
git show bed3de7 -- contracts/contracts/verifiers/

```

The commands above were used to avoid relying only on commit messages. For example, broad commits such as update were inspected using git show, git diff, and git blame so that the report could identify the actual files, functions, scripts, and bug fixes introduced by the contributor.

Appendix B – Evidence Repositories

The contribution analysis considered the integrated RollupX repository and the connected component repositories. These repositories were reviewed because some components were initially developed separately and later integrated into the main project repository.

- Personal GitHub profile: <https://github.com/UchihaItachi>
- Project organization: <https://github.com/RollupX-FYP>
- Main integrated repository: <https://github.com/RollupX-FYP/rollupx-full-zk-rollup>
- Benchmark suite repository: <https://github.com/RollupX-FYP/benchmark-suite>
- Submitter repository: <https://github.com/RollupX-FYP/submitter>
- Smart contracts repository: <https://github.com/RollupX-FYP/contracts>
- Data tools repository: <https://github.com/RollupX-FYP/data-tools>

Appendix C – Code Contribution Map and Evidence Tables

This section summarizes the key evidence, input-output data flows, bug fixes, and exact file mappings proving my individual technical workload.

TABLE A.1: Table A — Individual Code Contribution Map

Area	File	What I Implemented	Technical Importance
Submitter	<code>orchestrator.rs</code>	Batch processing loop and state transitions.	Enabled automated L1 batch settlement.
Submitter	<code>ethereum.adapter.rs</code>	BridgeClient trait and Ethereum RPC integration.	Decoupled L1 interaction; allowed mock-testing DA strategies.
Submitter	<code>da_blob.rs</code> <code>da_calldata.rs</code>	DA specific commitment generation and metadata encoding.	Provided configurable DA methods for empirical comparison.
Contracts	<code>ZKRollupBridge.sol</code>	<code>commitBatch</code> logic, DA routing, Verifier routing.	Handled L1 batch finality and state root updates.
Benchmark	<code>wait_for_sequencer.sh</code>	API readiness polling script for the Sequencer.	Resolved Docker startup race conditions.
Benchmark	<code>poisson_generator.py</code>	Exponential inter-arrival delay logic for workload.	Allowed mathematically rigorous synthetic TPS generation.
Data	<code>aggregate.py</code>	Batch ID joining and metric extraction.	Enabled end-to-end latency calculations from raw logs.

TABLE A.2: Table B — Input / Output Code Flow

Component	Input	Main Code Path	Output	Downstream Consumer
Submitter Flow	Finalized Batch	<code>Orchestrator</code> → <code>DaStrategy</code>	Tx: Hash & Soft Confirmation	Data tools pipeline
Smart Contract	Batch Payload	<code>commitBatch</code> → <code>provider.validateDA</code> → <code>verifier.verifyProof</code>	BatchCommitment Event	LE Explorers / Rollup Monitors
Data Aggregation	JSONL Logs	<code>aggregate.py</code> → merges by <code>batch_id</code>	Merged CSV Metrics	<code>stats.py</code> / Evaluators

TABLE A.3: Table C — Verified Bug Fixes

Bug	Root Cause	Code Fix	File	Impact on Benchmark Validity
False Positive Submissions	A mined L1 tx could be a reverted execution, but was marked success.	Added receipt-status check (<code>status == 1</code>) before confirmation.	<code>da_calldata.rs</code> & <code>da_blob.rs</code>	Prevented failed batches from silently corrupting benchmark metrics.
Startup Race Condition	Workload generator fired before Sequencer API was fully ready.	Added a polling loop (<code>wait_for_sequencer.sh</code>) blocking on <code>/health</code> .	<code>wait_for_sequencer.sh</code>	Stabilized automated benchmarking, preventing random failures.
Address Parsing Error	Executor crashed on 0x-prefixed addresses from generator.	Implemented <code>strip_prefix("0x")</code> fallback parsing.	<code>service.rs</code>	Allowed seamless integration between benchmark suite and rust Executor.

TABLE A.4: Evidence Strength Summary

Claim	Evidence Source	Notes
Implemented Submitter core workflow	Commit e4dad07; orchestrator.rs; Submitter infrastructure modules	The commit history and source structure show contribution to the Submitter orchestration, DA strategy flow, bridge interaction, and settlement handling.
Introduced L1 bridge abstraction	Commit e4dad07; ethereum_adapter.rs; BridgeClient trait	The abstraction decouples Submitter logic from direct Ethereum RPC calls and supports mock-based testing of L1 submission paths.
Implemented receipt-status validation	Commit e4dad07; da_calldata.rs; da_blob.rs	The source code contains explicit receipt-status checking using <code>status.as_u64() == 1</code> , preventing reverted L1 transactions from being treated as successful.
Contributed L1 bridge and DA contracts	Commit bed3de7; ZKRollupBridge.sol; DA and verifier-related contracts	The contract files provide the local L1 settlement boundary used by the Submitter and benchmark pipeline.
Automated benchmark execution	Commit 9b282f0; run_experiment.sh; run_matrix.sh; experiments.toml	The scripts and configuration files support repeated benchmark execution, service setup, workload triggering, and metric extraction.
Added service-readiness handling	Commit 9b282f0; wait_for_sequencer.sh	The readiness check prevents workload generation from starting before the relevant service endpoint is available.
Built workload generation support	Commit 9b282f0; poisson_generator.py	The workload generator provides controlled synthetic transaction input for evaluating batching and scheduling behaviour.
Built data analysis tooling	Commit bc90d40; aggregate.py; stats.py; plotting scripts	The data-tools pipeline converts raw benchmark outputs into aggregated datasets, statistics, and evaluation plots.
Executor/Sequencer internal ownership	Mixed commits; selected parsing/configuration evidence only	The report only claims verified parsing, configuration, or integration fixes. It does not claim full ownership of shared Executor or Sequencer internals.